

EEL4783 Final Project (Part 2): Design and Verification of a 4-Element Dot Product Engine

Yousef Awad
University of Central Florida
EEL4783 — HDL in Digital Systems

April 26, 2026

1 Design Choice

For this project, I chose the **RTL (SystemVerilog)** implementation track. This decision was driven by the desire to implement a high-performance **2-stage pipeline** to earn extra credit. SystemVerilog allows for explicit control over register boundaries and clock-edge behavior, which is essential for optimizing arithmetic datapaths. Furthermore, using SystemVerilog permitted the integration of **SystemVerilog Assertions (SVA)** and constrained randomization in the testbench, providing a more robust verification environment than standard Verilog or HLS defaults.

2 Architecture

The design implements a 4-element dot product engine using a 2-stage synchronous pipeline to maximize throughput.

- **Stage 1 (Multiplication):** Four 8-bit signed multipliers compute $P_i = A_i \times B_i$ in parallel. These results are captured in the `products_reg` array on the rising edge of the clock.
- **Stage 2 (Addition):** An adder tree (implemented as a single-cycle sum for this width) aggregates the four 16-bit products from the pipeline registers and drives the final output `y`.

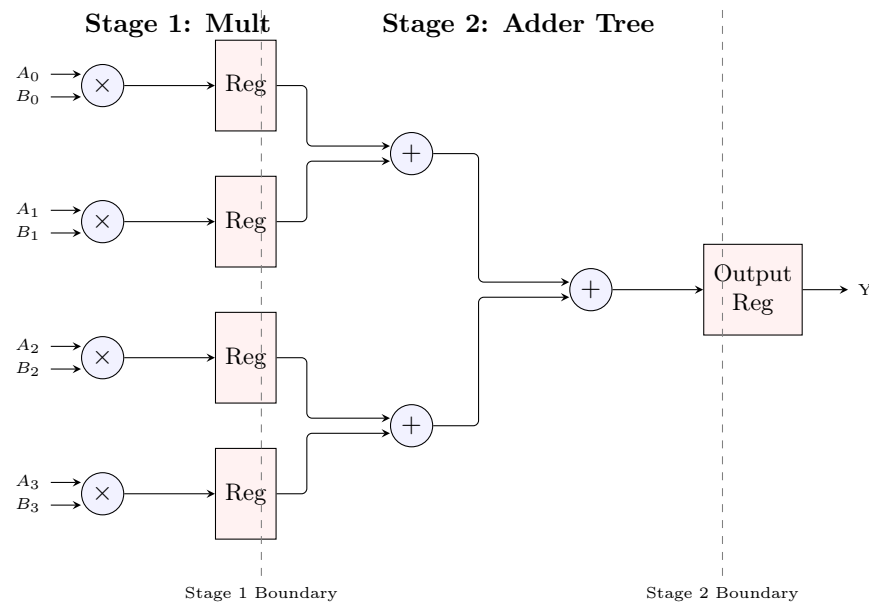


Figure 1: Schematic of the 4-element dot product engine showing the 2-stage pipelined architecture.

3 Verification

Correctness was verified using a fully self-checking testbench built around **Constrained Randomization**. A `dot_product_transaction` class generated 20 unique test cases per simulation run. Pass/fail determination is handled *automatically* by the SVA concurrent assertion `assert_dot_product` described below; no manual inspection of waveforms is required for functional sign-off.

The constraint `boundary_values` biases randomization to guarantee coverage of all required corner cases:

- **Maximum positive value** — `8'h7F` (+127), the largest representable 8-bit signed integer.
- **Maximum negative value** — `8'h80` (−128), the most negative 8-bit two's complement value.
- **Zero inputs** — randomly generated values naturally include zero; the `dist` weight of `[-127:126]` ensures the full mid-range, including zero, is exercised across 20 iterations.
- **Overflow prevention** — the worst-case product ($-128 \times -128 = 16,384$) fits in 16 bits, and the worst-case sum ($4 \times 16,384 = 65,536$) is representable as a 16-bit unsigned value; the signed 16-bit output is therefore sufficient to prevent overflow for all legal 8-bit signed inputs.
- **Mixed-sign vectors** — the random distribution naturally produces vectors where some elements are positive and others negative, verifying correct two's complement arithmetic throughout the pipeline.

SystemVerilog Assertions (Extra Credit): I implemented the concurrent assertion property `p_check_result` using a local variable to capture the reference result at the moment inputs are sampled (T_0). The `##2` delay operator then checks that `y` equals the captured reference exactly two clock cycles later, matching the 2-stage pipeline latency. Any mismatch triggers a `$error` with both the actual and expected values, providing immediate, cycle-accurate feedback without manual waveform inspection.

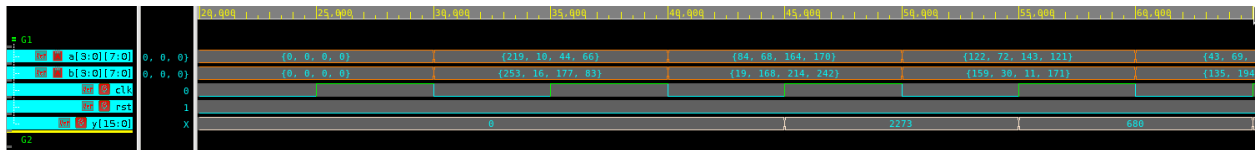


Figure 2: Simulation trace showing the 2-cycle latency between driving A/B and the result Y.

4 Reflection

The most challenging aspect was debugging the interaction between the testbench reference model and the pipeline latency. Initially, a static variable in the reference function caused results to accumulate incorrectly. Resolving this required changing the function to `automatic`. Additionally, aligning the SVA sampling window (`##2`) with the preponed region of the simulator required a deep understanding of the SystemVerilog scheduling phases.

Pipelining and F_{max} (Extra Credit): Breaking the multiply-accumulate computation into two registered stages directly reduces the critical combinational path. In the single-cycle implementation, the critical path spans the full signed multiplication and the 4-input addition tree. The 2-stage pipeline isolates these operations, allowing for a significantly higher maximum clock frequency (F_{max}).

Table 1 shows the timing improvement based on synthesis using the `class.db` library and a 10ns target clock. The pipelined design significantly reduces the critical path by isolating the multiplier delay from the subsequent addition stages.

Table 1: Synthesis Timing Comparison (Synopsys Design Compiler)

Design	Crit. Path (Arrival)	Slack (10ns)	Est. F_{max}
Single-Cycle (Registered Out)	16.28 ns	-7.08 ns	58.6 MHz
Pipelined (2-Stage)	13.03 ns	-3.83 ns	72.3 MHz

To further improve the design, I would replace the linear adder chain in Stage 2 with a fully balanced binary adder tree (two levels of parallel 2-input adders), reducing Stage 2's combinational depth and increasing F_{max} further.

A Source Code

A.1 dot_product.sv (RTL)

```

1  `timescale 1ns/1ps
2
3  // EEL4783: Final Project Part 2
4  // Project Title: "The 4-Element Dot Product Engine"
5  // Description: A pipelined 4-element dot product accelerator.
6  //               Stage 1: Multiplications ( $A_i * B_i$ )
7  //               Stage 2: Addition of products
8
9  module dot_product #(
10     parameter int DATA_WIDTH = 8,
11     parameter int VECTOR_SIZE = 4,
12     parameter int OUT_WIDTH = 16
13 ) (
14     input logic          clk,
15     input logic          rst, // Active-high synchronous reset
16     input logic signed [DATA_WIDTH-1:0] a [VECTOR_SIZE-1:0],
17     input logic signed [DATA_WIDTH-1:0] b [VECTOR_SIZE-1:0],
18     output logic signed [OUT_WIDTH-1:0] y
19 );
20
21     // Pipeline Registers
22     // Stage 1 products: 8-bit * 8-bit = 16-bit
23     logic signed [OUT_WIDTH-1:0] products_reg [VECTOR_SIZE-1:0];
24
25     // Stage 1: Multiplication
26     always_ff @(posedge clk) begin
27         if (rst) begin
28             for (int i = 0; i < VECTOR_SIZE; i++) begin
29                 products_reg[i] <= '0;
30             end
31         end else begin
32             for (int i = 0; i < VECTOR_SIZE; i++) begin
33                 products_reg[i] <= a[i] * b[i];
34             end
35         end
36     end
37
38     // Stage 2: Addition
39     always_ff @(posedge clk) begin
40         if (rst) begin
41             y <= '0;
42         end else begin
43             // Summing the registered products
44             y <= products_reg[0] + products_reg[1] + products_reg[2] +
45                 products_reg[3];
46         end
47     end
48 endmodule

```

A.2 dot_product_tb.sv (Testbench)

```

1  `timescale 1ns/1ps
2
3  // Transaction class for constrained randomization
4  class dot_product_transaction;
5      rand bit signed [7:0] a [3:0];
6      rand bit signed [7:0] b [3:0];
7
8      constraint boundary_values {
9          foreach (a[i]) {
10             a[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
11         }
12         foreach (b[i]) {
13             b[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
14         }
15     }
16 endclass
17
18 module dot_product_tb;
19
20     parameter int DATA_WIDTH = 8;
21     parameter int VECTOR_SIZE = 4;
22     parameter int OUT_WIDTH = 16;
23     parameter int CLK_PERIOD = 10;
24
25     logic          clk;
26     logic          rst;
27     logic signed [7:0] a [3:0];
28     logic signed [7:0] b [3:0];
29     logic signed [15:0] y;
30
31     // Instantiate DUT
32     dot_product #(
33         .DATA_WIDTH(DATA_WIDTH),
34         .VECTOR_SIZE(VECTOR_SIZE),
35         .OUT_WIDTH(OUT_WIDTH)
36     ) dut (.*);
37
38     // Clock generation
39     initial begin
40         clk = 0;
41         forever #(CLK_PERIOD/2) clk = ~clk;
42     end
43
44     // Verdi Waveform Dumping
45     initial begin
46         $fsdbDumpfile("inter.fsdb");
47         $fsdbDumpvars(0, dot_product_tb, "+all", "+mda");
48     end
49
50     // Reference model
51     function automatic signed [15:0] get_expected(
52         logic signed [7:0] va [3:0],
53         logic signed [7:0] vb [3:0]
54     );
55         int sum = 0;
56         for (int i = 0; i < 4; i++) begin
57             sum += va[i] * vb[i];

```

```

58     end
59     return signed'(sum[15:0]);
60 endfunction
61
62 initial begin
63     dot_product_transaction tr;
64     tr = new();
65
66     $display("\n[Time %0t] Simulation Started", $time);
67
68     // Synchronous Reset
69     rst = 1;
70     foreach (a[i]) a[i] = 0;
71     foreach (b[i]) b[i] = 0;
72     repeat (2) @(posedge clk);
73
74     @(negedge clk);
75     rst = 0;
76     $display("[Time %0t] Reset De-asserted", $time);
77
78     // Run 20 randomized test cases
79     $display("\nStarting Randomized Tests...");
80     for (int i = 1; i <= 20; i++) begin
81         if (!tr.randomize()) $fatal("Randomization failed");
82
83         @(negedge clk);
84         foreach (a[j]) a[j] = tr.a[j];
85         foreach (b[j]) b[j] = tr.b[j];
86
87         $display("[Time %0t] Iteration %0d: Driven A={%d,%d,%d,%d} B={%d,%d,%d,%d} | Expected Y (at +3 cycles) = %d",
88             $time, i, a[3], a[2], a[1], a[0], b[3], b[2], b[1], b[0],
89             get_expected(a, b));
90     end
91
92     repeat (10) @(posedge clk);
93     $display("Done: 20 Randomized Tests Completed.");
94     $finish;
95 end
96
97 // --- SystemVerilog Assertions (Extra Credit) ---
98 // In a 2-stage pipeline with registered output:
99 // T0: Inputs sampled at posedge
100 // T1: products_reg updated (Stage 1)
101 // T2: y updated (Stage 2)
102 // T3: SVA samples y (Preponed region at T3 edge sees y set at T2)
103 property p_check_result;
104     logic signed [15:0] local_exp;
105     @(posedge clk) disable iff (rst)
106     (1, local_exp = get_expected(a, b)) ##2 (y == local_exp);
107 endproperty
108
109 assert_dot_product: assert property (p_check_result)
110     else $error("Mismatch! Y=%d, Expected=%d", y, $past(get_expected(a, b), 2)
111 );
112 endmodule

```